iOS Application Development Overview

Sammani Maddumahewage
Lecturer (Probationary)
BSc (Hons) in Information Technology

What is the iOS Ecosystem?

The **iOS ecosystem** refers to the complete environment in which iOS applications are **designed, developed, tested, distributed, and maintained**.

Unlike Android, iOS is a **closed and controlled ecosystem**, fully managed by Apple.

This ecosystem includes:

- Apple hardware
- iOS operating system
- Development tools
- Programming languages
- UI frameworks
- App distribution channels
- Apple services

Apple Hardware and Devices

iOS apps run only on Apple devices such as:

- iPhone
- iPad
- iPod Touch

Key characteristics:

- Limited device variations
- High hardware–software optimization
- Predictable screen sizes and performance

Impact on developers: Less device fragmentation compared to Android → easier testing and UI consistency.

iOS Operating System Architecture

iOS follows a **layered architecture**:

1. **Cocoa Touch**
    - UIKit
    - SwiftUI
    - User interaction
2. **Media**
    - Audio, video, graphics
3. **Core Services**
    - Data, networking, location
4. **Core OS**
    - Memory, security, file system

Developers mainly interact with **Cocoa Touch layer**.

App Distribution Model

Apps are distributed **only via Apple App Store**

Strict review process

Requires Apple Developer Account

Emphasis on:

- Security
- Privacy
- UI consistency

Xcode and iOS Project Structure

What is Xcode?

Xcode is Apple's official **Integrated Development Environment (IDE)** for:

- iOS
- iPadOS
- macOS
- watchOS
- tvOS

Core Features of Xcode

- Swift code editor
- Interface Builder (Storyboard & SwiftUI)
- iOS Simulator
- Debugger (LLDB)
- Performance analyzer
- Asset management

Creating an iOS Application (UIKit)

When creating a project in Xcode, developers must select:

- Interface: **Storyboard**
- Language: **Swift**
- Life Cycle: **UIKit App Delegate**

This creates a UIKit-based application.

iOS Project Structure Explained

File	Explanation
AppDelegate.swift	Handles application-level events
SceneDelegate.swift	Manages UI scenes
ViewController.swift	Controls a screen
Main.storyboard	Visual UI layout
Assets.xcassets	Images, icons
Info.plist	App configuration

AppDelegate.swift (Entry Point)

```
import UIKit

@main
class AppDelegate: UIResponder, UIApplicationDelegate {
}
```

Role:

- App launch
- App termination
- Global app behavior

Comparable to **Android Application class**.

SceneDelegate.swift

Introduced for multi-window support.

```
class SceneDelegate: UIResponder, UIWindowSceneDelegate {
    var window: UIWindow?
}
```

Role:

- Controls UI lifecycle
- Sets initial ViewController

Swift Language Fundamentals

Why Swift?

Swift is:

- Safe
- Fast
- Modern
- Readable

Designed to avoid common runtime crashes.

Variables and Constants

var score = 90

let maxScore = 100

`var`: value can change

`let`: value is fixed

Type Inference vs Explicit Types

let name = "Kamal" // inferred

let age: Int = 21 // explicit

Swift encourages type safety.

Optionals (Important Concept)

var email: String?

Optional means:

- Value may exist
- Or may be nil

Safe access:

```
if let userEmail = email {

    print(userEmail)

}
```

Functions in Swift

```
func calculateSum(a: Int, b: Int) -> Int {
    return a + b
}
```

Classes and Initializers

```
class User {
    var username: String
    init(username: String) {
        self.username = username
    }
```

ViewControllers and Lifecycle

What is a ViewController?

A **ViewController**:

- Controls one screen
- Handles UI logic
- Manages user interaction

Equivalent to **Android Activity**.

Basic ViewController Code

```
import UIKit

class LoginViewController: UIViewController {


    override func viewDidLoad() {
        super.viewDidLoad()
        print("Login Screen Loaded")
    }
}
```

ViewController Lifecycle Flow

`viewDidLoad()`

`viewWillAppear()`

`viewDidAppear()`

`viewWillDisappear()`

Storyboards (UIKit UI Development)

What is a Storyboard?

A **Storyboard** is a **visual UI layout file** where:

- Screens are drawn
- Navigation is designed visually
- UI elements are connected to code

UI Components Used

- UILabel
- UITextField
- UIButton

Connecting UI to Code

IBOutlet (UI → Code)

```
@IBOutlet weak var usernameField: UITextField!
```

IBAction (User action → Code)

```
@IBAction func loginButtonPressed(_ sender: UIButton) {
    print("Login clicked")
}
```

Simple Login Validation Example

```
@IBAction func loginTapped(_ sender: UIButton) {
    if usernameField.text == "admin" {
        print("Login Success")
    } else {
        print("Login Failed")
    }
}
```

Navigation Using Segues

- Control + Drag
- Choose **Show**
- Assign Segue Identifier

```
performSegue(withIdentifier: "homeSegue", sender: self)
```

SwiftUI Overview (Modern UI Framework)

What is SwiftUI?

SwiftUI is a **declarative UI framework** where:

- UI is written as code
- State controls UI
- Less boilerplate

SwiftUI Structure

```
struct ContentView: View {
    var body: some View {
        Text("Hello iOS")
    }
}
```

SwiftUI Login UI Example

```
struct LoginView: View {
    @State private var username = ""

    var body: some View {
        VStack {
            TextField("Username", text: $username)
                .textFieldStyle(.roundedBorder)

            Button("Login") {
                print(username)
            }
        }
        .padding()
    }
}
```

SwiftUI vs UIKit

Feature	UIKit	SwiftUI
UI	Storyboard	Code
State Handling	Manual	Automatic
Complexity	High	Low

Thank You!